基于 Pytorch 的深度学习实践 (六)

YYFL

March 2026

1 多模态大模型概述

多模态大模型旨在统一处理文本、图像、语音等不同类型的信息，核心是将不同模态的输入映射到共享的向量空间，实现跨模态的理解与生成。其中，Vision Transformer (ViT) 解决了图像的 Transformer 编码问题，CLIP (Contrastive Language-Image Pre-Training) 则实现了图像与文本的跨模态对齐，是多模态领域的里程碑模型。

2 ViT：图像的 Transformer 编码

2.1 核心思想

ViT 将图像切分为固定大小的 Patch（如 16×16），每个 Patch 展平为一维向量后，通过线性投影转换为 Token（类似 Transformer 的词嵌入），再加入位置编码，最终输入标准 Transformer Encoder 进行编码。

2.2 关键步骤

1. **图像分块**：将尺寸为 $H \times W \times C$ 的图像切分为 N 个 $P \times P \times C$ 的 Patch，其中 $N = \frac{H \times W}{P^2}$（例如 224×224 图像切分为 16×16 Patch，得到 196 个 Patch）。
2. **Patch 嵌入**：每个 Patch 展平为 $P^2 \times C$ 向量，通过线性层投影到 d_{model} 维（如 768 维），得到 Patch Embedding。
3. **位置编码**：与 Transformer 文本位置编码一致，可为固定正弦余弦编码或可学习编码，加到 Patch Embedding 中。

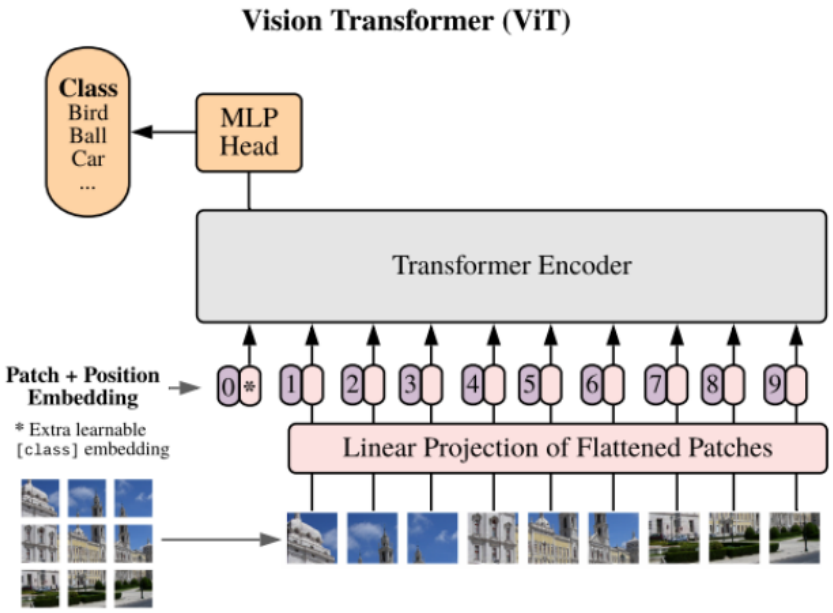

图 1: ViT 架构

4. **类别 Token**：在 Patch Embedding 前添加一个特殊的 <cls> Token，其最终输出用于图像分类任务。

5. **Transformer Encoder**：由多头自注意力（Multi-Head Attention）和前馈网络（FFN）组成，输出 <cls> Token 作为图像的全局特征。

2.3 ViT 代码示例（PyTorch）

```
import torch
import torch.nn as nn

class PatchEmbedding(nn.Module):
    def __init__(self, img_size=224, patch_size=16, in_chans=3, embed_dim=768):
        super().__init__()
        self.img_size = img_size
```

```python
        self.patch_size = patch_size
        self.num_patches = (img_size // patch_size) **2
        # 卷积实现分块+线性投影
        self.proj = nn.Conv2d(in_chans, embed_dim, kernel_size=patch_size, stride=patch_size)

    def forward(self, x):
        # x: (batch_size, 3, 224, 224)
        x = self.proj(x)  # (batch_size, 768, 14, 14)
        x = x.flatten(2)  # (batch_size, 768, 196)
        x = x.transpose(1, 2)  # (batch_size, 196, 768)
        return x

class ViT(nn.Module):
    def __init__(self, img_size=224, patch_size=16, in_chans=3, embed_dim=768, num_heads=12, num_layers=12):
        super().__init__()
        # Patch Embedding
        self.patch_embed = PatchEmbedding(img_size, patch_size, in_chans, embed_dim)
        # 类别Token
        self.cls_token = nn.Parameter(torch.zeros(1, 1, embed_dim))
        # 位置编码
        self.pos_embed = nn.Parameter(torch.zeros(1, self.patch_embed.num_patches + 1, embed_dim))
        # Transformer Encoder
        encoder_layer = nn.TransformerEncoderLayer(d_model=embed_dim, nhead=num_heads, batch_first=True)
        self.encoder = nn.TransformerEncoder(encoder_layer, num_layers=num_layers)
        # 分类头
        self.head = nn.Linear(embed_dim, 1000)  # ImageNet 1000类

    def forward(self, x):
        batch_size = x.shape[0]
        # Patch Embedding
        x = self.patch_embed(x)  # (batch_size, 196, 768)
```

```python
        # 添加类别Token
        cls_token = self.cls_token.expand(batch_size, -1, -1)  # (batch_size, 1, 768)
        x = torch.cat([cls_token, x], dim=1)  # (batch_size, 197, 768)
        # 添加位置编码
        x = x + self.pos_embed
        # Transformer编码
        x = self.encoder(x)
        # 取cls_token输出
        cls_out = x[:, 0]
        # 分类
        logits = self.head(cls_out)
        return logits

# 测试
if __name__ == "__main__":
    model = ViT()
    x = torch.randn(2, 3, 224, 224)  # batch_size=2
    out = model(x)
    print(out.shape)  # (2, 1000)
```

3 CLIP：跨模态对比学习

CLIP (Contrastive Language-Image Pre-Training) 由 OpenAI 提出，通过对比学习将图像和文本映射到同一向量空间，实现了强大的零样本（Zero-Shot）图像分类能力。

3.1 CLIP 整体架构

CLIP 包含两大核心模块：

1. **图像编码器**：可选 ViT（如 ViT-B/32）或 ResNet，输出图像的全局特征向量。
2. **文本编码器**：采用 Decoder-only 架构的 Transformer（基于 BPE 分词），输出文本的全局特征向量（<|endoftext|> Token 或 <cls> Token）。

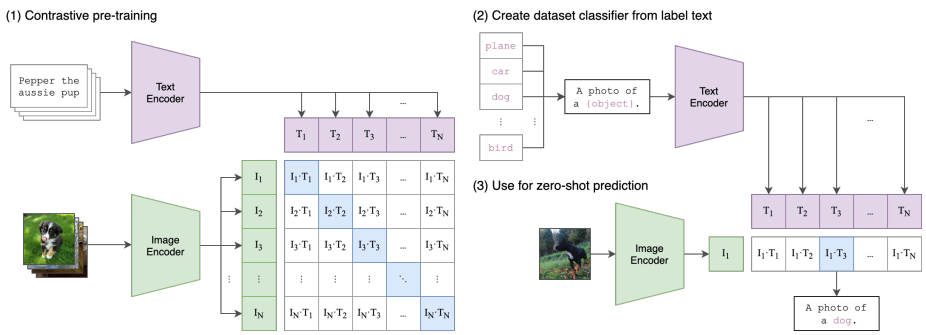

图 2: CLIP 架构

3. **投影层**：图像和文本编码器的输出分别通过独立的线性层投影到相同维度（如 512 维），并进行 L2 归一化，使向量落在单位超球面上。

3.2 对比损失函数

CLIP 的核心是对比学习，目标是最大化“图像-文本正样本对”的相似度，最小化“图像-文本负样本对”的相似度。

假设批次中有 N 个图像-文本对，构建 $N \times N$ 的相似度矩阵 $\mathbf{S}$，其中 $\mathbf{S}_{i,j}$ 表示第 i 张图像与第 j 段文本的余弦相似度（归一化后点积）。

对比损失分为图像侧和文本侧：

$$\mathcal{L}_{\text{img}} = -\frac{1}{N}\sum_{i=1}^{N}\log\frac{\exp(\mathbf{S}_{i,i}/\tau)}{\sum_{j=1}^{N}\exp(\mathbf{S}_{i,j}/\tau)}$$

$$\mathcal{L}_{\text{text}} = -\frac{1}{N}\sum_{j=1}^{N}\log\frac{\exp(\mathbf{S}_{j,j}/\tau)}{\sum_{i=1}^{N}\exp(\mathbf{S}_{i,j}/\tau)}$$

$$\mathcal{L}_{\text{CLIP}} = \frac{\mathcal{L}_{\text{img}} + \mathcal{L}_{\text{text}}}{2}$$

其中 τ 为温度系数，用于调节相似度的分布。

3.3 CLIP 的零样本能力

传统图像分类模型只能识别训练集中见过的类别，而 CLIP 可通过“文本提示工程”实现零样本分类：

1. 构造目标类别的文本提示（如 ‘”a photo of a cat”‘、‘”a photo of a dog”‘）。
2. 将文本输入文本编码器得到文本向量，图像输入图像编码器得到图像向量。
3. 计算图像向量与所有文本向量的余弦相似度，取最大者为分类结果。

3.4 CLIP 代码示例（HuggingFace）

```python
from transformers import CLIPProcessor, CLIPModel
import torch

# 加载预训练模型和处理器
model = CLIPModel.from_pretrained("openai/clip-vit-base-patch32")
processor = CLIPProcessor.from_pretrained("openai/clip-vit-base-patch32")

# 准备数据：图像和候选文本描述
image_path = "example.jpg"  # 替换为你的图像路径
candidate_texts = [
    "a photo of a cat",
    "a photo of a dog",
    "a photo of a bird",
    "a photo of a car"
]

# 预处理：图像和文本统一编码
inputs = processor(
    text=candidate_texts,
    images=image_path,
    return_tensors="pt",
    padding=True
)

# 前向传播（禁用梯度）
with torch.no_grad():
    outputs = model(**inputs)
```

```python
    # 图像-文本相似度分数 (logits_per_image: [1, 4])
    logits_per_image = outputs.logits_per_image
    # 转换为概率
    probs = logits_per_image.softmax(dim=1)

# 输出结果
max_prob_idx = probs.argmax().item()
print(f"最匹配的描述: {candidate_texts[max_prob_idx]}")
print(f"置信度: {probs[0][max_prob_idx]:.4f}")
# 输出所有候选的概率
for i, text in enumerate(candidate_texts):
    print(f"{text}: {probs[0][i]:.4f}")
```

4 多模态大模型的扩展与应用

4.1 扩展方向

1. **LLaVA**：将 CLIP 的图像特征与大语言模型（如 LLaMA）结合，实现图像问答（VQA）。
2. **Stable Diffusion**：基于 CLIP 的文本编码器，将文本转换为图像特征，驱动扩散模型生成图像。
3. **多模态生成**：统一处理文本、图像、语音，实现图文生成、语音转文字等任务。

4.2 典型应用场景

- 零样本图像分类：无需标注数据，直接通过文本提示分类新类别。
- 跨模态检索：输入文本检索相似图像，或输入图像检索相似文本。
- 视觉问答（VQA）：根据图像回答自然语言问题。
- 图文生成：根据文本描述生成图像，或根据图像生成文本摘要。